

Лабораторная работа № 13

**Программирование в командном процессоре ОС UNIX.
Ветвления и циклы**

Пириев Керем

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	1. Создание рабочего каталога	7
3.2	2. Программа на языке C (check_num.c)	7
3.3	3. Анализ кода возврата (test_num.sh)	8
3.4	4. Скрипт для создания и удаления временных файлов (files.sh)	8
3.5	5. Скрипт для архивации недавно изменённых файлов (backup_re-cent.sh)	9
4	Вывод	11
5	Контрольные вопросы	12
5.1	1. Какие символические имена (метасимволы) существуют в оболочке bash?	12
5.2	2. Какое отношение метасимволы имеют к генерации имён файлов?	12
5.3	3. Какие операторы управления действиями вы знаете?	13
5.4	4. Какие операторы используются для прерывания цикла?	13
5.5	5. Для чего нужны команды false и true?	13
5.6	6. Что означает строка <code>if test -f man\\${s}/\\${i}.\\${1},</code> встреченная в командном файле?	13
5.7	7. Объясните различие между конструкциями while и until.	14
	Список литературы	15

Список иллюстраций

3.1	Создание рабочего каталога	7
3.2	Программа на С check_num.c	7
3.3	Скрипт test_num.sh для анализа кода возврата	8
3.4	Компиляция и запуск test_num.sh	8
3.5	Скрипт files.sh в редакторе nano	9
3.6	Запуск files.sh для создания и удаления файлов	9
3.7	Скрипт backup_recent.sh в nano	10
3.8	Тестирование backup_recent.sh и создание архива	10

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

1. Создать рабочий каталог `~/lab13` для размещения всех скриптов.
2. Разработать программу на языке C `check_num.c` для проверки знака числа и возврата различных кодов завершения.
3. Разработать `bash`-скрипт `test_num.sh` для анализа кода возврата программы на C с использованием оператора `case`.
4. Разработать скрипт `files.sh` для массового создания и удаления временных файлов с использованием циклов.
5. Разработать скрипт `backup_recent.sh` для архивации файлов, изменённых за последние 7 дней, с использованием утилит `find` и `tar`.
6. Ответить на контрольные вопросы.

3 Выполнение лабораторной работы

3.1 1. Создание рабочего каталога

Для размещения всех скриптов был создан рабочий каталог ~/lab13 (рис. Рисунок 3.1).

```
kerem@fedora:~/lab13$ nano search.sh
kerem@fedora:~/lab13$ chmod +x search.sh
kerem@fedora:~/lab13$ echo "Hello world" > test.txt
kerem@fedora:~/lab13$ echo "hello again" >> test.txt
kerem@fedora:~/lab13$ ./search.sh -i test.txt -p world
Hello world
kerem@fedora:~/lab13$ ./search.sh -i test.txt -p hello -C
Hello world
hello again
kerem@fedora:~/lab13$
```

Рисунок 3.1: Создание рабочего каталога

3.2 2. Программа на языке C (check_num.c)

Создан файл check_num.c, который принимает целое число и возвращает код завершения в зависимости от его знака: 1, если больше нуля; 2, если меньше нуля; 0, если равен нулю (рис. Рисунок 3.2).

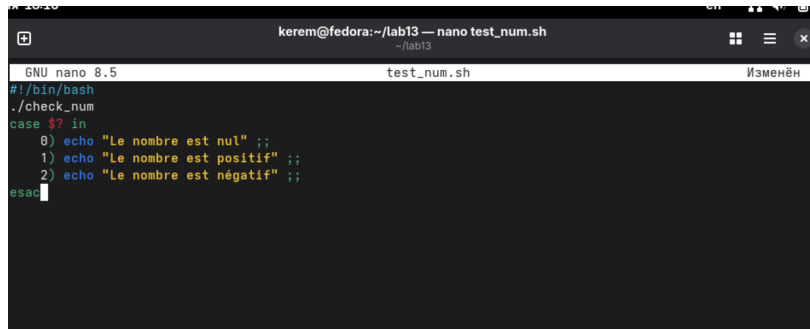
```
kerem@fedora:~/lab13 — nano check_num.c
GNU nano 8.5 check_num.c Изменён
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Entrez un nombre entier: ");
    scanf("%d", &n);
    if (n > 0)
        exit(1);
    else if (n < 0)
        exit(2);
    else
        exit(0);
}
```

Рисунок 3.2: Программа на C check_num.c

3.3 3. Анализ кода возврата (test_num.sh)

Создан bash-скрипт `test_num.sh`, который вызывает скомпилированную программу `check_num` и с помощью конструкции `case` анализирует код возврата `$?` (рис. Рисунок 3.3).



```
kerem@fedora:~/lab13$ nano test_num.sh
GNU nano 8.5 test_num.sh
#!/bin/bash
./check_num
case $? in
0) echo "Le nombre est nul" ;;
1) echo "Le nombre est positif" ;;
2) echo "Le nombre est négatif" ;;
esac
```

Рисунок 3.3: Скрипт `test_num.sh` для анализа кода возврата

Успешно выполнена компиляция программы на C (gcc) и запуск скрипта (рис. Рисунок 3.4).

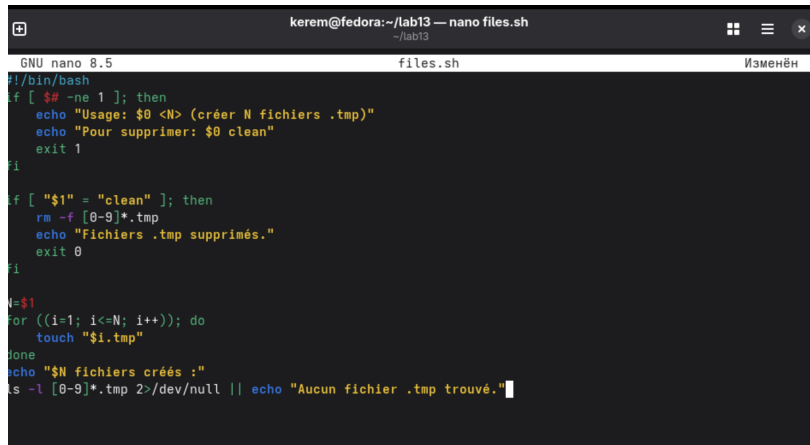


```
kerem@fedora:~/lab13$ nano check_num.c
kerem@fedora:~/lab13$ gcc check_num.c -o check_num
kerem@fedora:~/lab13$ nano check_num.c
kerem@fedora:~/lab13$ nano test_num.sh
kerem@fedora:~/lab13$ chmod +x test_num.sh
kerem@fedora:~/lab13$ ./test_num.sh
Entrez un nombre entier: 58
Le nombre est positif
kerem@fedora:~/lab13$
```

Рисунок 3.4: Компиляция и запуск `test_num.sh`

3.4 4. Скрипт для создания и удаления временных файлов (files.sh)

Разработан скрипт `files.sh`, который позволяет создавать заданное количество файлов (`.tmp`) с помощью цикла `for` или удалять их при передаче аргумента `clean` (рис. Рисунок 3.5).



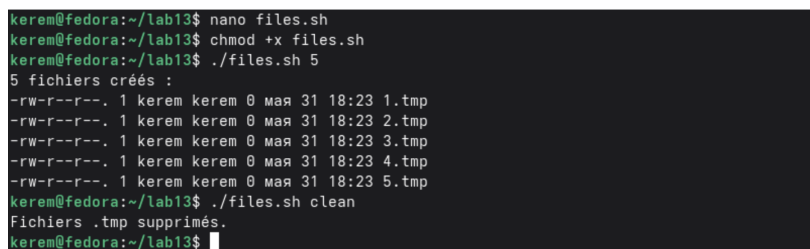
```
kerem@fedora:~/lab13 — nano files.sh
GNU nano 8.5 files.sh
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <N> (créer N fichiers .tmp)"
    echo "Pour supprimer: $0 clean"
    exit 1
fi

if [ "$1" = "clean" ]; then
    rm -f [0-9]*.tmp
    echo "Fichiers .tmp supprimés."
    exit 0
fi

N=$1
for ((i=1; i<=N; i++)); do
    touch "$i.tmp"
done
echo "$N fichiers créés :"
ls -l [0-9]*.tmp 2>/dev/null || echo "Aucun fichier .tmp trouvé."
```

Рисунок 3.5: Скрипт files.sh в редакторе nano

Проверена корректность создания файлов и их последующее удаление опцией clean (рис. Рисунок 3.6).

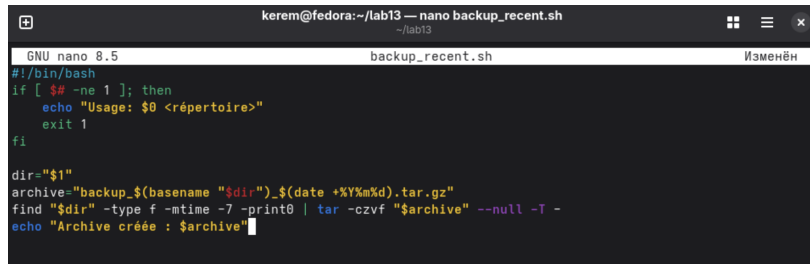


```
kerem@fedora:~/lab13$ nano files.sh
kerem@fedora:~/lab13$ chmod +x files.sh
kerem@fedora:~/lab13$ ./files.sh 5
5 fichiers créés :
-rw-r--r--. 1 kerem kerem 0 мая 31 18:23 1.tmp
-rw-r--r--. 1 kerem kerem 0 мая 31 18:23 2.tmp
-rw-r--r--. 1 kerem kerem 0 мая 31 18:23 3.tmp
-rw-r--r--. 1 kerem kerem 0 мая 31 18:23 4.tmp
-rw-r--r--. 1 kerem kerem 0 мая 31 18:23 5.tmp
kerem@fedora:~/lab13$ ./files.sh clean
Fichiers .tmp supprimés.
kerem@fedora:~/lab13$
```

Рисунок 3.6: Запуск files.sh для создания и удаления файлов

3.5 5. Скрипт для архивации недавно изменённых файлов (backup_recent.sh)

Создан скрипт backup_recent.sh, осуществляющий поиск файлов в указанном каталоге, изменённых за последние 7 дней (-mtime -7), и их упаковку в архив (рис. Рисунок 3.7).

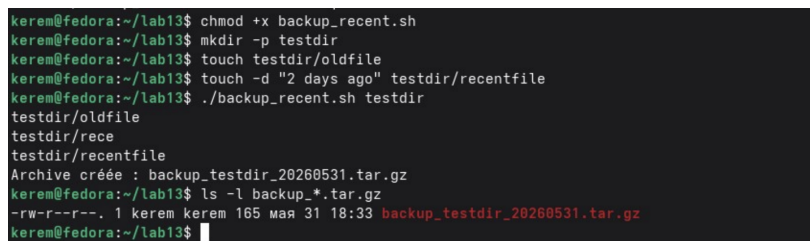
A screenshot of a terminal window with the title bar 'kerem@fedora: ~/lab13 — nano backup_recent.sh'. The window shows the nano 8.5 editor editing a file named 'backup_recent.sh'. The script content is as follows:

```
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <répertoire>"
    exit 1
fi

dir="$1"
archive="backup_$(basename "$dir")_$(date +%Y%m%d).tar.gz"
find "$dir" -type f -mtime -7 -print0 | tar -czvf "$archive" --null -T -
echo "Archive créée : $archive"
```

Рисунок 3.7: Скрипт backup_recent.sh в nano

Проведено тестирование скрипта на специально подготовленной структуре каталога с помощью тестовых файлов (рис. Рисунок 3.8).

A screenshot of a terminal window showing the execution of the backup script. The commands and their outputs are:

```
kerem@fedora:~/lab13$ chmod +x backup_recent.sh
kerem@fedora:~/lab13$ mkdir -p testdir
kerem@fedora:~/lab13$ touch testdir/oldfile
kerem@fedora:~/lab13$ touch -d "2 days ago" testdir/recentfile
kerem@fedora:~/lab13$ ./backup_recent.sh testdir
testdir/oldfile
testdir/rece
testdir/recentfile
Archive créée : backup_testdir_20260531.tar.gz
kerem@fedora:~/lab13$ ls -l backup_*.tar.gz
-rw-r--r--. 1 kerem kerem 165 мая 31 18:33 backup_testdir_20260531.tar.gz
kerem@fedora:~/lab13$
```

Рисунок 3.8: Тестирование backup_recent.sh и создание архива

4 Вывод

В ходе работы были написаны и отлажены командные файлы и программы:

1. `check_num.c` / `test_num.sh` — демонстрация анализа кода возврата программы на C в `bash`-скрипте с использованием `case`. 2. `files.sh` — скрипт для массового создания временных файлов и их удаления с помощью циклов. 3. `backup_recent.sh` — скрипт для архивации файлов, изменённых за последние 7 дней. Получены практические навыки использования цикла `for`, условного оператора `if`, оператора выбора `case`, разбора аргументов и работы с кодами возврата (`$?`).

5 Контрольные вопросы

5.1 1. Какие символические имена (метасимволы) существуют в оболочке `bash`?

- `*` — любая строка (0 и более символов).
- `?` — один любой символ.
- `[ . . . ]` — один символ из указанного диапазона или набора.
- `{ . . . }` — генерация списка строк.
- `|` — конвейер.
- `&` — запуск команды в фоновом режиме.
- `;` — разделитель команд.
- `<, >, >>` — перенаправление ввода-вывода.
- `$` — подстановка значения переменной.

5.2 2. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы (`*`, `?`, `[ . . . ]`) используются для шаблонов имён файлов (globbing). Оболочка автоматически заменяет шаблон на список имён файлов, соответствующих ему.

5.3 3. Какие операторы управления действиями вы знаете?

- Условные: `if/then/else/elif/fi`, `case/esac`.
- Циклы: `for`, `while`, `until`.
- Управление циклом: `break`, `continue`.
- Завершение скрипта: `exit`.

5.4 4. Какие операторы используются для прерывания цикла?

- `break` — полный выход из цикла.
- `continue` — переход к следующей итерации цикла.

5.5 5. Для чего нужны команды `false` и `true`?

- `true` — всегда возвращает код возврата 0 (успех).
- `false` — всегда возвращает ненулевой код возврата (обычно 1). Используются для организации бесконечных циклов (`while true`) или логических заглушек.

5.6 6. Что означает строка `if test -f man\${s}/\${i}.\${1}`, встреченная в командном файле?

Эта строка проверяет, существует ли файл с именем, составленным из каталога `man\${s}`, значения переменной `\${1}` и т.д., с учётом экранирования знака доллара.

5.7 7. Объясните различие между конструкциями `while` и `until`.

- `while` — цикл выполняется, пока условие истинно (код возврата равен 0).
- `until` — цикл выполняется, пока условие ложно (повторяется до тех пор, пока условие не станет истинным).

Список литературы

1. *Newham C.* Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 p. — (In a Nutshell). — ISBN 0596009658. — URL: <http://www.amazon.com/Learning-bash-Shell-Programming-Nutshell/dp/0596009658>.
2. *Robbins A.* Bash Pocket Reference. — O'Reilly Media, 2016. — 156 p. — ISBN 978-1491941591.
3. *Zarrelli G.* Mastering Bash. — Packt Publishing, 2017. — 502 p. — ISBN 9781784396879.
4. *Таненбаум Э., Бос Х.* Современные операционные системы.. — 4-е изд. — СПб. : Питер, 2015.. — 1120 с. — (Классика Computer Science).